Paravirt Scheduling:
Framework for better CPU usage

Shrikanth Hegde Ilya Leoshkevich
OSPM 2026, Cambridge, IBM

Agenda

- Problem Statement and proposed Solution
- Existing methods
- Journey so far
- Design goals
- Implementation details
- s390 background
- Performance numbers
- Open questions

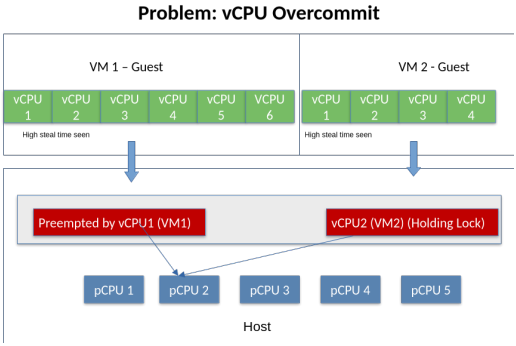
© 2026 IBM Corporation

1. **Problem Statement and Solution**
2. Existing methods
3. Journey so far
4. Design goals
5. Implementation details
6. s390 background
7. Performance numbers
8. Open questions

© 2026 IBM Corporation

Problem Statement: vCPU Overcommit and vCPU Preemption

- Multiple VMs sharing the underlying CPU resources, with overcommit (sum of vCPUs > pCPUs)
- High CPU utilization in multiple VMs at the same time leads to vCPU preemption
- Why vCPU preemption can be expensive:
  - **vCPU context:** a vCPU may be holding a mutex or spinlock, or running with interrupts disabled, with other vCPUs waiting on it
  - **Hypervisor overhead:** hypervisors may schedule at a different granularity than the guest, introducing extra delays and cache/TLB thrashing on vCPU migration



© 2026 IBM Corporation

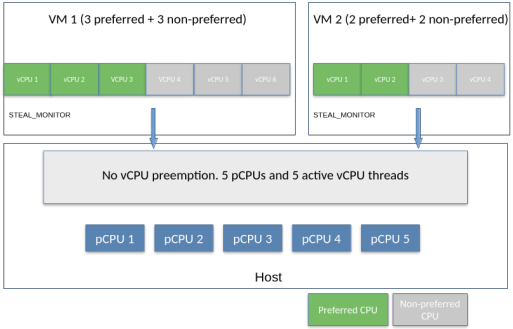
Proposed Solution: steal-driven vCPU backoff

- Every guest does proportional back-off
- The amount of vCPUs to use is driven by steal time
- The vCPUs each guest can safely use are denoted as *preferred* CPUs
- Aim: sum of preferred CPUs $\leq$ pCPUs, which achieves less vCPU preemption

Advantage of a context switch within the vCPU instead:

- Lockholder runs to completion (`preempt_disable()`)
- No hypervisor overhead

Solution: cpu_preferred_mask (Steal-driven Backoff)



© 2026 IBM Corporation

1 Problem Statement and Solution

2 **Existing methods**

3 Journey so far

4 Design goals

5 Implementation details

6 s390 background

7 Performance numbers

8 Open questions

© 2026 IBM Corporation

Existing Methods

Discussed in detail at LPC 2025 – why no existing methods work. A quick glance:

- **Explicit affinity:** can't expect a user to set affinity for each workload
- **CPU hotplug:** slow, expensive, breaks user-specified affinities
- **Isolated cpuset partitions:** needs sched domain rebuild, breaks user-specified affinities
- **Capacity-aware scheduling:** doesn't push tasks from a CPU under high load

© 2026 IBM Corporation

1 Problem Statement and Solution

2 Existing methods

3 **Journey so far**

4 Design goals

5 Implementation details

6 s390 background

7 Performance numbers

8 Open questions

© 2026 IBM Corporation

Journey so far...

- **LPC 2024** – introduced the problem statement. A new load balancing type `cpu_parked` was used. This was a PoC, slow in moving tasks since it depended on the load balancer and heavily on stats.
  [RFC PATCH 0/2] sched/fair: introduce new scheduler group type `group_parked`
- **LPC 2025:** a few revisions later, switched to a push-task-mechanism-based method. Much faster and more policy-driven than stats.
  [PATCH 00/17] Paravirt CPUs and push task for less vCPU preemption

© 2026 IBM Corporation

1 Problem Statement and Solution

2 Existing methods

3 Journey so far

4 **Design goals**

5 Implementation details

6 s390 background

7 Performance numbers

8 Open questions

© 2026 IBM Corporation

Design Goals

- Keep runnable vCPUs close to pCPU count under contention
- Preserve user-specified CPU affinity
- Keep the solution scheduler-native and lightweight
- Provide a debug tunable to adjust based on the platform
- Expose the new CPU state information for consumption

© 2026 IBM Corporation

1 Problem Statement and Solution

2 Existing methods

3 Journey so far

4 Design goals

5 **Implementation details**

6 s390 background

7 Performance numbers

8 Open questions

© 2026 IBM Corporation

New CPU State: `cpu_preferred_mask`

- Represents CPUs the scheduler should currently prefer
- Not a hard exclusion: affinity is still respected
- Dynamic and policy-driven
- Maintains the design construct:
  preferred $\leq$ online $\leq$ possible

[PATCH v2 00/17] sched/paravirt: Introduce `cpu_preferred_mask` and steal-driven vCPU backoff

© 2026 IBM Corporation

Sched feature: STEAL_MONITOR

- In `sched_tick()`, check if it is time to compute steal time
- Kick off a work function to compute steal time – keeps `sched_tick()` latency minimally affected
- Compute steal time ratio from `CPUTIME_STEAL` across all online CPUs
- If steal time is above the threshold, reduce preferred CPUs by one core. Core granularity is used because some hypervisors (e.g. PowerVM) schedule at core granularity – this yields the best benefit
- If steal time is below the threshold, increase preferred CPUs by one core
- Simple heuristics avoid oscillations: only consecutive high/low steal values are considered

© 2026 IBM Corporation

Core APIs being updated

- `is_cpu_allowed()`
  - Try to select a preferred CPU
  - If task affinity is only on non-preferred CPUs, return a non-preferred CPU
  - Ensure overhead is minimal for the majority use case
  - Used by `select_fallback_rq()`, the push-task mechanism, and wake-ups
- `available_idle_cpu()`
  - Make the CPU available if it is part of preferred CPUs
  - Handles wake-up decisions; used by CFS
- `__cpupri_find()`
  - Consider only preferred CPUs when finding `lowest_rq` for RT tasks

© 2026 IBM Corporation

How it all fits together

- On task wake-up, when choosing a CPU, pick a preferred CPU where possible
- Load-balance only among preferred CPUs, so load balance and push-task don't fight each other; load stays on preferred CPUs
- At `sched_tick()`, the first check: is this CPU non-preferred? If so, push the task out via a stopper thread. Only the current task is pushed (keeps it simple). Since the work is not done in `sched_tick()`, IRQ latency shouldn't increase much
- A task is pushed only if it doesn't break affinity. If a task is bound only on non-preferred CPUs, there is no point pushing it out

© 2026 IBM Corporation

User Visibility & Control

- `/sys/devices/system/cpu/preferred` – current preferred CPUs
- `STEAL_MONITOR` sched feature (disabled by default)
- Debugfs knobs for thresholds and sampling period

```
# echo STEAL_MONITOR > /sys/kernel/debug/sched/features
# echo NO_STEAL_MONITOR > /sys/kernel/debug/sched/features

# pwd
/sys/kernel/debug/sched
steal_mon_high:500
steal_mon_low:200
steal_mon_period:1000
```


© 2026 IBM Corporation

1 Problem Statement and Solution

2 Existing methods

3 Journey so far

4 Design goals

5 Implementation details

6 **s390 background**

7 Performance numbers

8 Open questions

© 2026 IBM Corporation

s390 challenges

- Large system: ~200 cores
- There is no bare-metal Linux on s390
- Proprietary type-1 hypervisor: PR/SM
  - Unlikely to add Linux-specific enhancements
- Overcommit and steal time
- Whole-system performance vs. single-VM performance

© 2026 IBM Corporation

s390 Hiperdispatch

- PR/SM designates CPUs as high, medium, and low, and lets the guest know which is which
- PR/SM frequently steals low CPUs for a long time
- Problem: exits to PR/SM are expensive
- Idea: avoid scheduling threads on low CPUs that had steal time recently

Hiperdispatch

- Created by Tobias Huschle and Mete Durlu
- Upstream since v6.12
- Uses capacity-aware scheduling
- Does not handle heavy overcommit

© 2026 IBM Corporation

s390 Warning Track Interrupt

- Problem: contention if a lock holder is stolen
- PR/SM sends an interrupt shortly before stealing a CPU
- Idea: park CPUs that are about to be stolen to prevent them from taking locks

WTI

- Created by Tobias Huschle and Mete Durlu
- Upstream since v6.12
- Schedules a real-time thread to yield to the hypervisor ASAP

© 2026 IBM Corporation

1 Problem Statement and Solution

2 Existing methods

3 Journey so far

4 Design goals

5 Implementation details

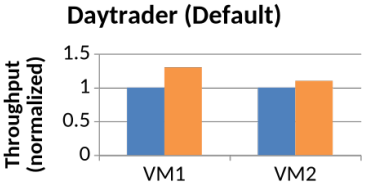
6 s390 background

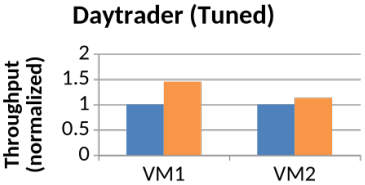
7 **Performance numbers**

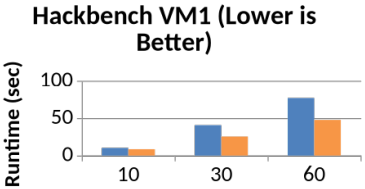
8 Open questions

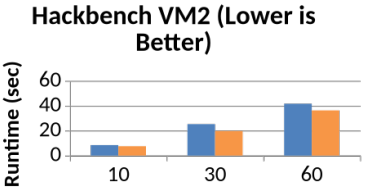
© 2026 IBM Corporation

Performance numbers: PowerPC / PowerVM hypervisor









© 2026 IBM Corporation

Performance numbers: s390 PR/SM

- s390 host with 37 cores
- 4 partitions with 37 shared cores each
- Workload 1: `stress-ng --matrix 37`
- Workload 2: `uperf` ping-pong
- Workload 3: `uperf` streaming

Results:

- ~2x throughput improvement for all workloads
- Steal time reduction from ~60% to ~5%

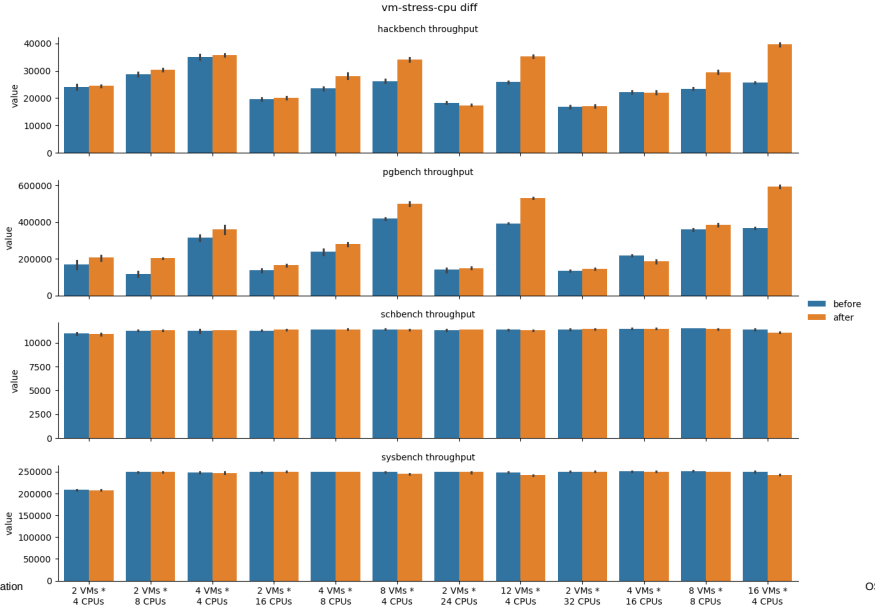
© 2026 IBM Corporation

Performance numbers: KVM

- 2–16 VMs
- 4–32 CPUs each
- s390 host with 16 cores
- x86_64 host with 32 cores
- Up to 4x overcommit
- Each VM runs the same benchmark
  - `hackbench`, `pgbench`, `schbench`, `sysbench`
- Throughputs are summed across all VMs
- Repeated 10 times to quantify noise

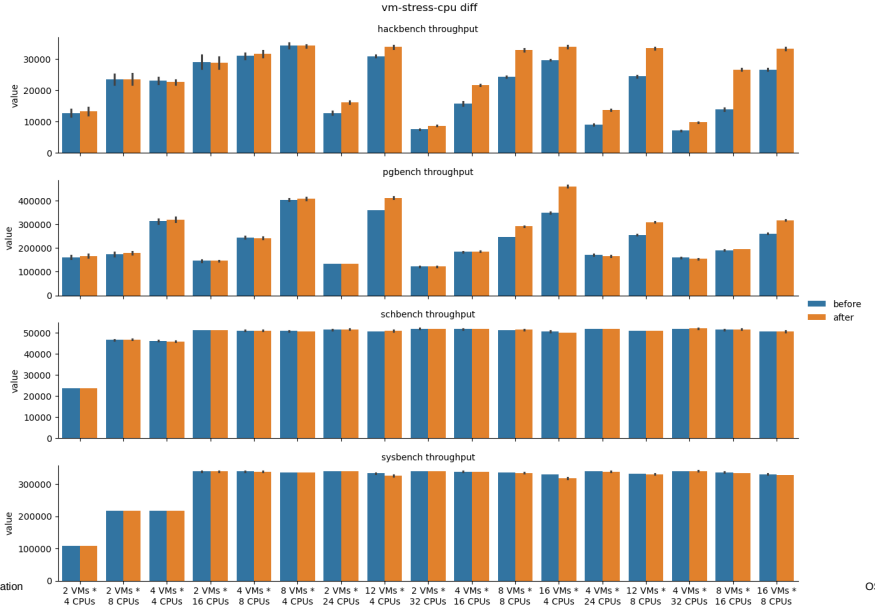
© 2026 IBM Corporation

Performance numbers: s390 KVM



© 2026 IBM Corporation

Performance numbers: x86_64 KVM



© 2026 IBM Corporation

1 Problem Statement and Solution

2 Existing methods

3 Journey so far

4 Design goals

5 Implementation details

6 s390 background

7 Performance numbers

8 **Open questions**

© 2026 IBM Corporation

Open Questions and Future Work

- Augment the generic policy with arch-specific hints, like power entitlements or s390 polarization. Arch-specific hooks in the steal-time monitor code?
- Interaction with `irqbalance`: make it preferred-mask-aware, or require it to be disabled?
- Is there any !SMT system interested in this?
- NUMA-awareness to avoid cross-node penalties
- `sched_ext`?
- How to avoid oscillations in complex overcommit cases?
- Should steal time be computed based on online CPUs or preferred CPUs?

© 2026 IBM Corporation

Legal statement

- This work represents the view of the authors and does not necessarily represent the view of IBM
- IBM and IBM (logo) are trademarks or registered trademarks of International Business Machines Corporation in the United States and/or other countries
- Linux is a registered trademark of Linus Torvalds
- Other company, product, and service names may be trademarks or service marks of others

© 2026 IBM Corporation

Thank You